kontaktstudium informatik koin

Themenbereich C
Lerneinheit C1
Übungsaufgaben

Hallo und herzlich willkommen zum ersten Übungsblatt zur Rechnerarchitektur.

In diesem und dem nächsten Blatt werden wir uns damit beschäftigen, wie Abläufe in kleinen Bauteilen, also konkret in Hardware, realisiert werden können. Dazu beschäftigen wir uns näher mit der Booleschen Algebra und logischen Ausdrücken, Wahrheitstabellen und Gatterschaltungen. Der Wechsel zwischen den verschiedenen Darstellungsformen der Booleschen Algebra ist eine Frage der Übung. Um eine gewisse Routine im Umgang mit der Schaltalgebra zu erlangen, konzentriert sich dieses Arbeitsblatt auf Anwendungsaufgaben.
Für die Erstellung von Gatterschaltungen gibt es mehrere Möglichkeiten. Falls keine Vorlage in Moodle vorgegeben ist, können Sie die Gatterschaltungen natürlich von Hand zeichnen. Eine andere Möglichkeit ist die Verwendung von Diagrams.net (früher Draw.io). Diagrams.net ist ein Tool zum Zeichnen von Grafiken, das auch im Browser funktioniert. Hier können Sie die Symbole für die Schaltungen importieren. Die Importdatei finden Sie in den Materialien. Bitte verwenden Sie für alle Optionen die IEC60617-12 Darstellung, die wir auch in den Materialien verwenden. Wir erwarten diese Darstellung in der Prüfung und werden für andere Darstellungen Punkte abziehen.

Wir wünschen Ihnen viel Spaß beim Übungsblatt!

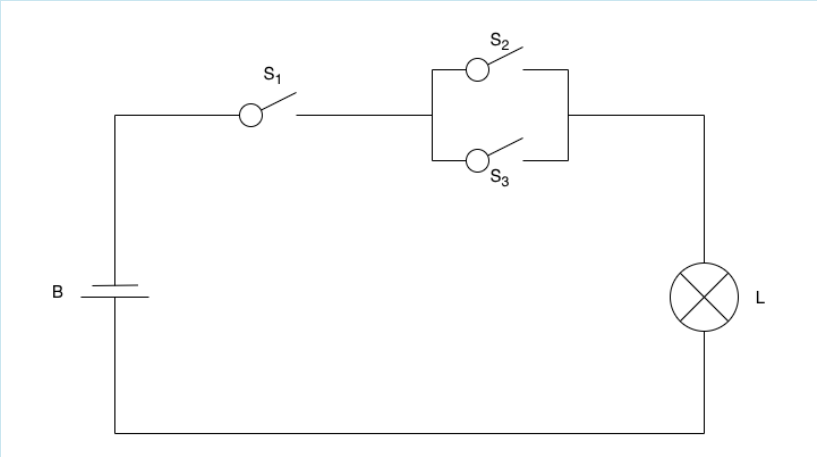
Transferaufgabe

Programmieraufgabe

Anwendungsaufgabe

Mögliche Klausuraufgabe

Aufgabe 1: Schaltkreise (3 Punkte)
Gegeben sei der folgende Schaltkreis mit einer Batterie B, den Schaltern S_1, S_2 und S_3 sowie der Lampe L.



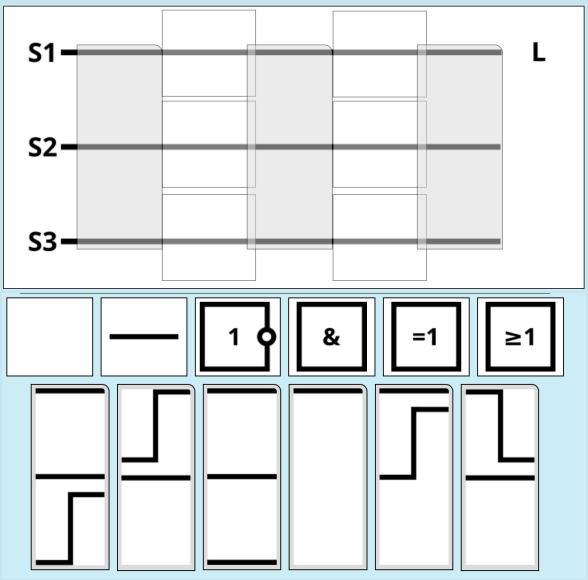
(1 Punkt) Leiten Sie von der gegebenen Schaltung eine Wahrheitstabelle ab. Geben Sie in der Wahrheitstabelle die Werte für die drei Schalter (S_1 - S_3) und für die Lampe (L) an. Zählen Sie dabei binär aufsteigend die Werte für die Schalter, wie in den ersten Zeilen gezeigt (000; 001; 010; ... -vgl. Wahrheitstabelle in Aufgabe 2).

S_1	S_2	S_3	L
0	0	0	
0	0	1	
0	1	0	

(1 Punkt) Welchem logischen Ausdruck entspricht die Schaltung von S1 - S3?
In Moodle stehen Ihnen hier die üblichen Operatoren $\wedge, \vee, \neg, \oplus$ zur Verfügung.

Hinweis: Für die Abgabe in Moodle bitte nur die nötigen Klammern und keine Leerzeichen verwenden. Die Subskripte einfach ersetzen: $S_1 = S1$, etc.

Überführen Sie den Schaltkreis bzw. die Wahrheitstabelle in eine Gatterschaltung. Befüllen Sie dafür **alle** Platzhalter, indem Sie die passenden Puzzleteile an die richtige Stelle ziehen (ggf. ein leeres Teil). Sie können auch erst eine Skizze von Hand anfertigen und diese dann hier nachbauen.



Aufgabe 2: Wahrheitstafeln (7 Punkte)

Stellen Sie zu folgendem Booleschen Ausdruck die Wahrheitstafel auf. Vereinfachen Sie den Ausdruck nicht. Wie in der Lerneinheit C1 gezeigt wird der Ausdruck zerlegt: die Zwischenschritte (Substitutionen) ergänzen Sie in der Wahrheitstafel passend. Geben Sie nicht nur binäre, sondern auch unnäre Operationen (Negation) an.

$$((a \vee b) \wedge \neg b) \wedge (b \vee (a \wedge \neg b))$$

a	b	$z = \neg b$	$x =$	$l = x \wedge z$	$y = a \wedge z$	$r =$	$l \wedge r$
0	0						
0	1						
1	0						
1	1						

Gegeben sei folgende Wahrheitstafel. Leiten Sie daraus die disjunktive Normalform D ab. Gehen Sie dabei genau so vor, wie in Lerneinheit C1 gezeigt.

a	b	c	y	Konjunktion
0	0	0	1	
0	0	1	0	
0	1	0	1	
0	1	1	0	
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	1	

$D =$

Hinweis: Für die Abgabe in Moodle bitte die Konjunktionen in der Tabelle ohne Klammern & Leerzeichen und lexikographisch geordnet (a, b, c) angeben. Die Disjunktionen nur mit den nötigen Klammern versehen und ebenfalls ohne Leerzeichen. Wählen Sie die Ordnung der Konjunktionen im finalen Ausdruck von oben nach unten, wie sie in der Tabelle auftauchen.

Zeigen Sie mit Hilfe von Wahrheitstafeln, dass folgende (semantische) Äquivalenz gilt. Ergänzen Sie dafür die benötigten Wahrheitstafeln wie in den Materialien der Lerneinheit C1 gezeigt inklusive aller Zwischenschritte (Substitutionen).

$$(a \vee b) \wedge c = (a \wedge c) \vee (b \wedge c)$$

$$l = (a \vee b) \wedge c$$

a	b	c	z =	l =
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

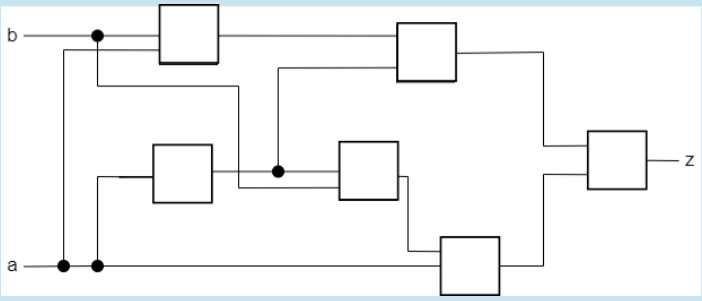
$$r = (a \wedge c) \vee (b \wedge c)$$

a	b	c	x =	y =	r =
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Aufgabe 3: Gatterschaltungen (5 Punkte)

Gegeben sei folgender Boolscher Ausdruck. Bilden Sie diesen wie in Lerneinheit C1 gezeigt in einem Gatternetz ab. Vereinfach Sie den Ausdruck **nicht**.

$$z = (a \wedge (\neg a \vee b)) \vee ((a \wedge b) \vee \neg a)$$



Gegeben sei das Gatter XOR. Wir wollen die Funktion dieses Gatters nur mit den Gattern AND, OR und NOT in einem Gatternetz abbilden. Verwenden Sie zur Ermittlung des benötigten Booleschen Ausdrucks eine Wahrheitstafel und geben Sie diese hier an.

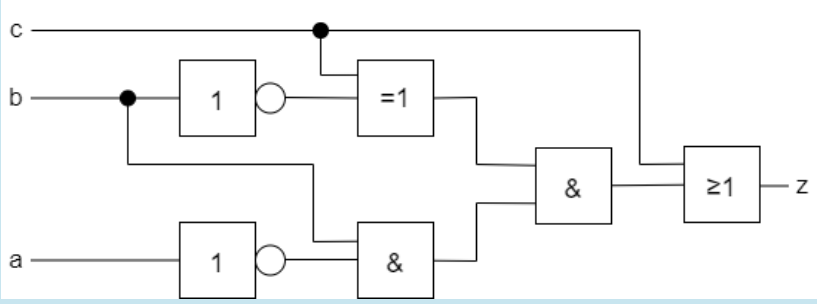
a	b	XOR	Konjunktion
0	0	0	
0	1	1	
1	0	1	
1	1	0	

$z =$

Hinweis: Für die Abgabe in Moodle bitte die Konjunktionen in der Tabelle ohne Klammern & Leerzeichen tippen. Die Disjunktionen nur mit den nötigen Klammern versehen und ebenfalls ohne Leerzeichen.

Erstellen Sie nun für das Gatter XOR ein Gatternetz nur mit den Gattern AND, OR und NOT und laden Sie es hier hoch. Der Boolsche Ausdruck der vorigen Teilaufgabe dient als Hilfestellung.
(Hier Datei in Moodle hochladen.)

Gegeben sei folgendes Gatternetz. Leiten Sie daraus den Booleschen Ausdruck ab. Gehen Sie dabei wie in Lerneinheit C1 beschrieben vor und beginnen Sie ganz rechts im Gatternetz. Der abgeleitete Ausdruck muss nicht vereinfacht werden.



$D =$

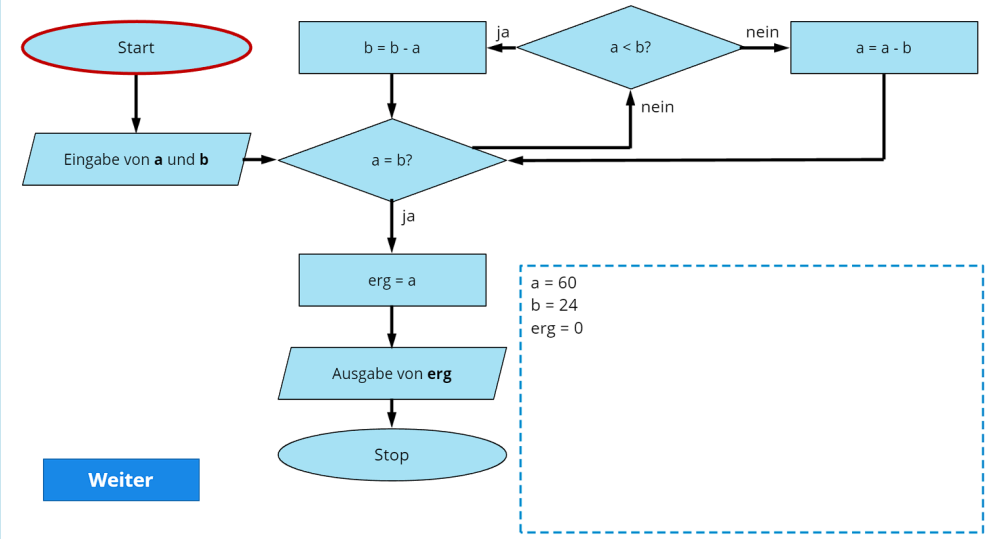
Hinweis: Für die Abgabe in Moodle bitte nur die zwingend nötigen Klammern und keine Leerzeichen verwenden.

Themenbereich D

Programmieraufgabe zu C1:
Bitte beachten Sie: Die folgende Aufgabe mit insgesamt 5 Punkten zählt zum Themenbereich D: Praktische Implementation von Algorithmen (vgl. Merkblatt zur Prüfungszulassung).

Aufgabe 4: Algorithmen implementieren (5 Punkte)

Gegeben sei der nachfolgende Algorithmus zur Berechnung des größten gemeinsamen Teilers von zwei positiven Ganzzahlen a und b, welcher durch ein Flussdiagramm beschrieben ist:



Führen Sie den Algorithmus für die Eingabe a = 60 und b = 24 durch. Notieren Sie jedes Mal, wenn der Algorithmus die Abfrage $a = b$? passiert, die Werte für a und b in folgender Tabelle. Geben Sie zum Schluss das Ergebnis **erg** an, das ausgegeben wird.

Durchlauf	a	b
	60	24
1		
2		
3		

erg:

Ein programmierbegeisterter Mathematikfan hat ein Java-Programm geschrieben, das beim Kürzen von Brüchen unterstützen soll. Den aktuellen Stand des Projekts sehen Sie in diesem Bild:

```java
//Dieses Programm soll beim Kürzen von Brüchen unterstützen.

//Array 1: enthält alle Zähler
int[] zaehler = {5, 8, 2, 9, 14, 60, 23};
//Array 2: enthält alle Nenner
int[] nenner = {15, 3, 22, 9, 67, 24, 11};

//in einer Schleife werden alle Brüche zusammengebaut
for(int i = 0; i < zaehler.length; i++) {
    //ungekürzter Bruch
    String bruchDarstellung = ("Ungekürzt: " + zaehler[i] + " / " + nenner[i]);
    System.out.println(bruchDarstellung);

    //berechnen des ggT
    int ichKuerzeMit = ggT(zaehler[i], nenner[i]);
    zaehler[i] = zaehler[i]/ichKuerzeMit;
    nenner[i] = nenner[i]/ichKuerzeMit;

    //gekürzter Bruch
    String bruchDarstellungNeu = ("Gekürzt: " + zaehler[i] + " / " + nenner[i]);
    System.out.println(bruchDarstellungNeu);

    System.out.println();
}
```

Sie können den Code außerdem in Moodle herunterladen, um ihn selbst zu testen.

Leider geht aktuell noch etwas schief! Im Programm soll für jeden Bruch der größte gemeinsame Teiler von Zähler und Nenner bestimmt werden, um anschließend damit zu kürzen. Es wird aber beim Ausführen des Codes folgende Fehlermeldung angezeigt:

```
jshell> /open LE_C1_ggT_brueche_kuerzen.jsh
|  Fehler:
|  Symbol nicht gefunden
|    Symbol: Methode ggT(int,int)
|    Ort: Klasse
|      int ichKuerzeMit = ggT(zaehler[i], nenner[i]);
|                         ^-^
|
```

Aktuell fehlt also noch die Methode mit dem Namen ***ggT***, die im gegebenen Code aufgerufen werden soll.

Ihre Aufgabe: Schreiben Sie eine Methode namens ***ggT***, mit der Sie das Programm erfolgreich ausführen könnten. Sie können die Methode zunächst in Visual Studio Code im vollständigen Code ergänzen und testen. Für die Abgabe in Moodle müssen Sie **nur die neue Methode** in das Abgabefenster kopieren. Mit einem Klick auf „Prüfen“ wird Ihre Methode für verschiedene Eingabeparameter getestet, die nicht direkt mit den Zahlen im Brüche-Programm zusammenhängen. Stellen Sie also sicher, dass Ihre Methode für beliebige ganzzahlige und positive Eingaben funktioniert, nicht nur für die in den Arrays gegebenen Beispielzahlen.

Achten Sie beim Implementieren der Methode besonders auf die folgenden syntaktischen und semantischen Details:

- Die Methode benötigt einen Rückgabetyp. Sie können aus dem gegebenen Programmcode herauslesen, welcher Datentyp hierfür erwartet wird.

- Der Name der Methode muss zum Programm passen.
- Die Eingabeparameter müssen zum Aufruf im Programm passen. Semantisch gesehen muss außerdem der berechnete ggT natürlich dem korrekten mathematischen Ergebnis entsprechen. Beim Umsetzen kann Ihnen das Flussdiagramm aus dem ersten Aufgabenteil helfen.

Zum Beispiel:

Test	Resultat
`System.out.println(ggT(45,15));`	15
`System.out.println(ggT(8,15));`	1

Antwort: (Abzugssystem: 0 %)

```
//Hier können Sie die ggT-Methode abgeben:
1
2
3
4
5
6
7
8
9
10
11
12
```